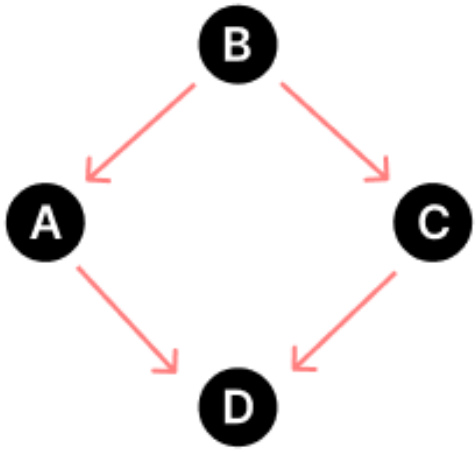
Data Structure & Algorithm 2 (Theory part)

1. **Every connected directed acyclic graph (DAG) has exactly one topological ordering" - is it true or false? Explain your answer briefly by designing a graph G with exactly 4 vertices that justify your reasoning.**

   Answer:
   The statement "Every connected directed acyclic graph (DAG) has exactly 1 topological order" is false.

   To show that not every connected DAG has exactly one topological ordering, we can design a graph G with exactly 4 vertices as follows:

   For example,



   This graph is a connected DAG with **4 vertices and 4 edges**. It has two possible topological orderings: **A, D, B, C**, and **A, D, C, B**.

   In general, a DAG can have multiple topological orderings if there are vertices that are not related to each other. In the example above, vertices B and C are not related to each other, so they can be ordered in different ways without affecting the topological ordering of the other vertices.

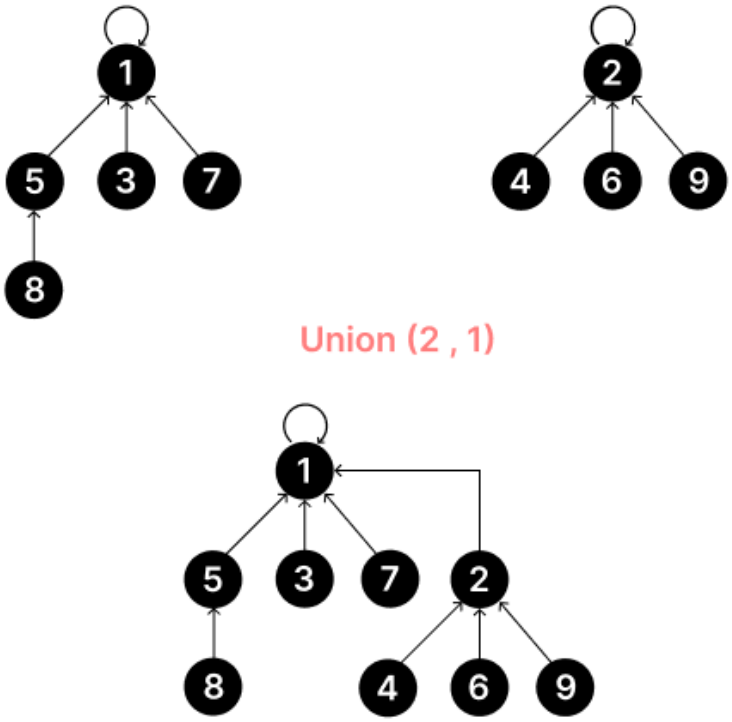
2. **What is the advantage of using the union-by-rank heuristic in a disjoint set? Explain your answer with an example.**

   Answer:
   Here are three advantages of using the union-by-rank heuristic in a disjoint set:

   - **Maintain a minimum height:** Union-by-rank maintains a minimum height where **the height of the resulting tree is always equal to the height of the bigger tree among the given tree.** But, if the given trees have the same height then the height of the resulting tree may vary.

   - **Easy to find:** The union-by-rank heuristic is more effective than the union-by-size heuristic because it ensures that **the depth of the resulting tree is minimized, which improves the time complexity of the find operation.**

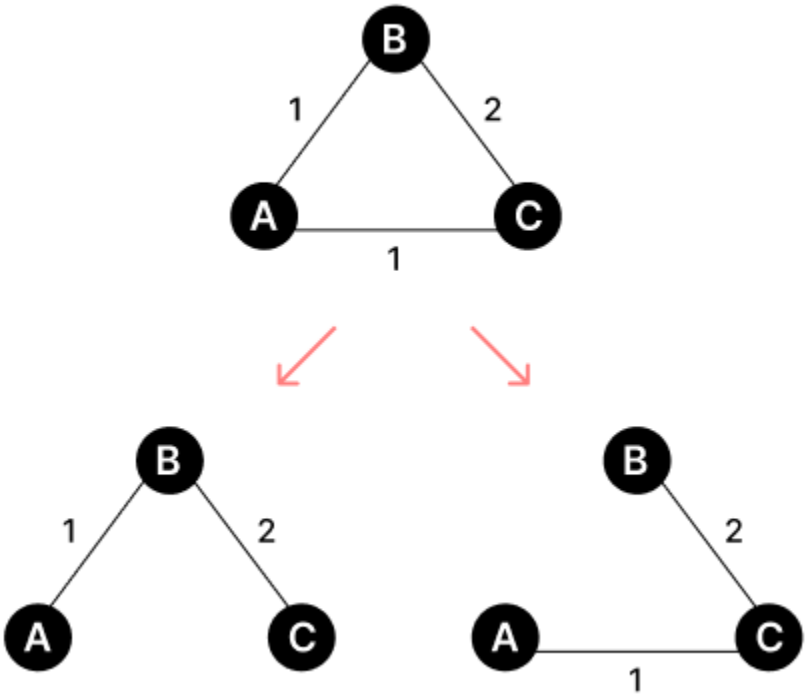
   For example,



3. **Suppose you are given a weighted graph where multiple edges have the same weight. You are asked to apply Kruskal's algorithm to find MST from that given graph. You have chosen the merge sort algorithm to sort the edges in ascending order. "Depending on the sequence of the sorted edges, you may have different MSTs". Is the statement True or False? Justify your answer.**

Answer:
The statement "Depending on the sequence of the sorted edges, you may have different MSTs" is true.

For example, suppose we have a weighted graph with three edges:
(A, B) with weight 1, (B, C) with weight 2, and (A, C) with weight 1.



If we sort the edges in ascending order of weight, we would get (A, B), (A, C), and (B, C). In this case, the MST would be {**(A, B), (B, C)**}.

However, if we sort the edges in a different order, such as (A, C), (A, B), and (B, C), we would get a different MST: {**(A, C), (A, B)**}.

4. **What will be the runtime complexity of Kruskal's algorithm if we use bubble sort to sort edges (Sorting n elements with bubble sort takes time $O(n$^2))?**

   Answer:

   Bubble sort has a time complexity of **O(n^2)**, where n is the number of elements. In Kruskal's algorithm, if we sort the edges in ascending order of weight using bubble sort, it will take **O(e^2)** time.

   Using merge sort, Kruskal's algorithm requires **O(e log e)**, where e is the number of edges in the graph.

   Therefore, the runtime complexity of the algorithm will be **O(e^2),** where e is the number of edges in the graph.

5. **A graph contains the vertices {1, 2, 3, 4, 5, 6, 7}, and the shortest path from 1 to 7 is 1→3→2→4→5→7→6. Is it possible to find the shortest path from 2 to 7 from the given data? Justify your answer.**

   Answer:

   Yes, it is possible to find the shortest path from 2 to 7 from the given data.

   The shortest path from 1 to 7 is 1→ 3→2→4→5→7→6 maintains an **Optimal Substructure Property** such that all the edges along this 1 to 7 path are already having the shortest path between them.

   So, we can just say that the shortest path from 2 to 7 will be,

   2 → 4 → 5 → 7

6. **Explain why a priority queue instead of a regular queue (FIFO) is used in Dijkstra's algorithm.**

   Answer:
   Dijkstra's algorithm is a shortest path algorithm that uses a priority queue instead of a regular queue (FIFO) to find the next node to explore.

   In Dijkstra's algorithm, each step requires a minimum weighted edge. Now, a min-priority queue is used as it **manages a list of values and gives priority to the element with the least value**. So, we do not need any loop/process to find the minimum weight in every step.

   But if we use a regular queue, we have to find out the least weight in every step which increases its runtime complexity. So, we use a priority queue instead of a regular queue (FIFO) is used in Dijkstra's algorithm.

7. **What are the advantages of open addressing vs chaining for collision resolution?**

   Answer:
   Open addressing and chaining are two methods used for collision resolution in hash tables. Here are the advantages of open addressing over chaining:

   - **Space efficiency:** Open addressing does not require additional space for pointers to store the keys outside the hash table, unlike chaining. This makes it more space-efficient, especially for small hash tables.

   - **Cache performance:** Open addressing has better cache performance than chaining because it stores all the elements in a contiguous block of memory. This reduces the number of cache misses and improves the overall performance of the hash table.

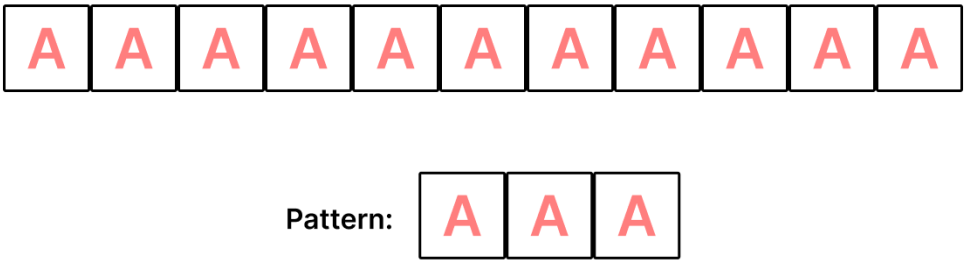
   - **Low load factor:** Open addressing is faster than chaining when the load factor is low because it does not require following pointers between list nodes. This makes it more efficient for hash tables with a low number of collisions.

   In general, open addressing is more space-efficient and has better cache performance than chaining. However, chaining may be more efficient for hash tables with a high number of collisions or when worst-case performance is a concern.

8. **Provide an example to demonstrate that the worst case of the Rabin Karp Algorithm can be**
   O((n – m+1) X m).

   Answer:
   The Rabin-Karp algorithm is a string-searching algorithm that uses hashing to find patterns in strings. The worst-case running time of the Rabin-Karp algorithm is O(nm), where n is the length of the text, and m is the length of the pattern, but in the worst case the time complexity can be **O((n-m+1) X m).**

   For example,



   Here, the Rabin-Karp algorithm would have a drawback as it needs **to calculate the hash values by doing a mod with a number for all the substrings of the text of length m, which will be the same as the hash value of the pattern "AAA".**
   Therefore, it would need to compare the pattern with each of these substrings, which takes **O(m) time**. And, the worst-case running time of the Rabin-Karp algorithm, in this case, would be **O((n-m+1) x m).**

   In general, the worst-case running time of the Rabin-Karp algorithm occurs when all the characters in the text and pattern are the same as the hash values of all the substrings of the text. This can happen when the hash function used by the algorithm is not good enough to avoid hash collisions.

9. **“Breadth-First Search” can create a minimum spanning tree for unweighted graphs. Explain why this statement is true.**

   Answer:
   The statement "Breadth-First Search (BFS) can create a minimum spanning tree for unweighted graphs" is true because,

   - If a graph is unweighted, or equivalently, we can consider that all edges have the same weight, then any spanning tree is a minimum spanning tree. Again, **BFS does not visit a node twice, and it only traverses from one vertex to another if it hasn't visited it before, such that there aren't going to be any cycles,** and if the source is connected somehow with all the vertices, it will eventually visit all nodes.

   So, we can use a BFS to find such a tree in time linear in the number of edges,

   For example,

10. **What is union-by-rank in the Disjoint-Set Union data structure? Why do we need this heuristic? Justify your answer.**

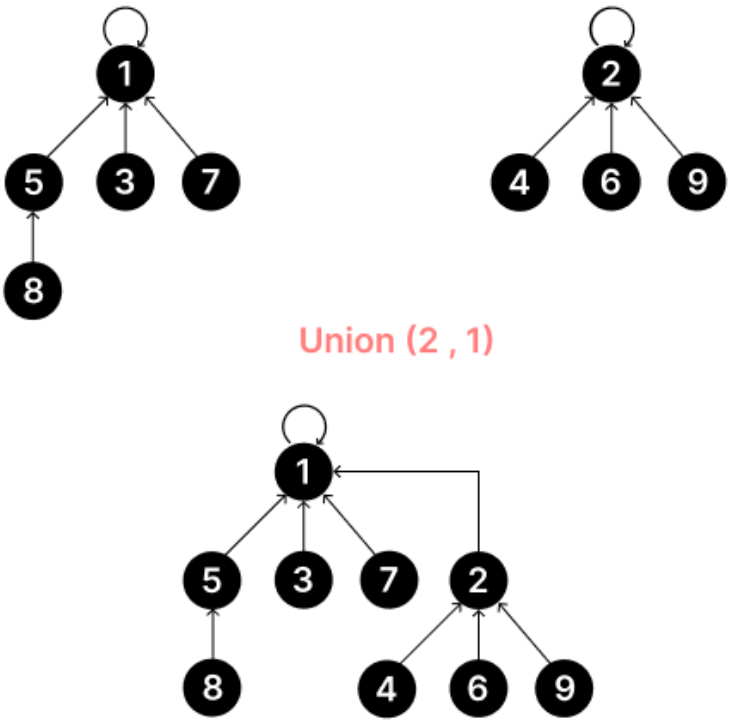
Answer:
Union-by-rank is a heuristic used in the Disjoint-Set Union data structure to always attach the smaller tree to the root of the larger tree during a union operation. We need this heuristic to reduce the time complexity of the find operation, which involves traversing the tree from a node to its root to find the set that the node belongs to.

Here are some reasons for using the union-by-rank heuristic in a disjoint set:

- **Maintain a minimum height:** Union-by-rank maintains a minimum height where **the height of the resulting tree is always equal to the height of the bigger tree among the given tree**. But, if the given trees have the same height then the height of the resulting tree may vary.

- **Easy to find:** The union-by-rank heuristic is more effective than the union-by-size heuristic because it ensures that **the depth of the resulting tree is minimized, which improves the time complexity of the find operation.**

For example,



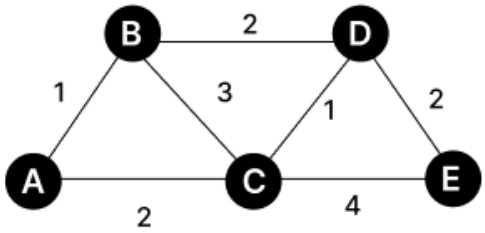
11. **Write the pseudocode of Union Sets and Find Sets operation of the Disjoint-Set Union data structure, assuming the path compression and union-by-rank heuristics.**

Answer:
Here is the pseudocode for the Union Sets and Find Sets operations of the Disjoint-Set Union data structure, assuming the path-compression and union-by-rank heuristics:

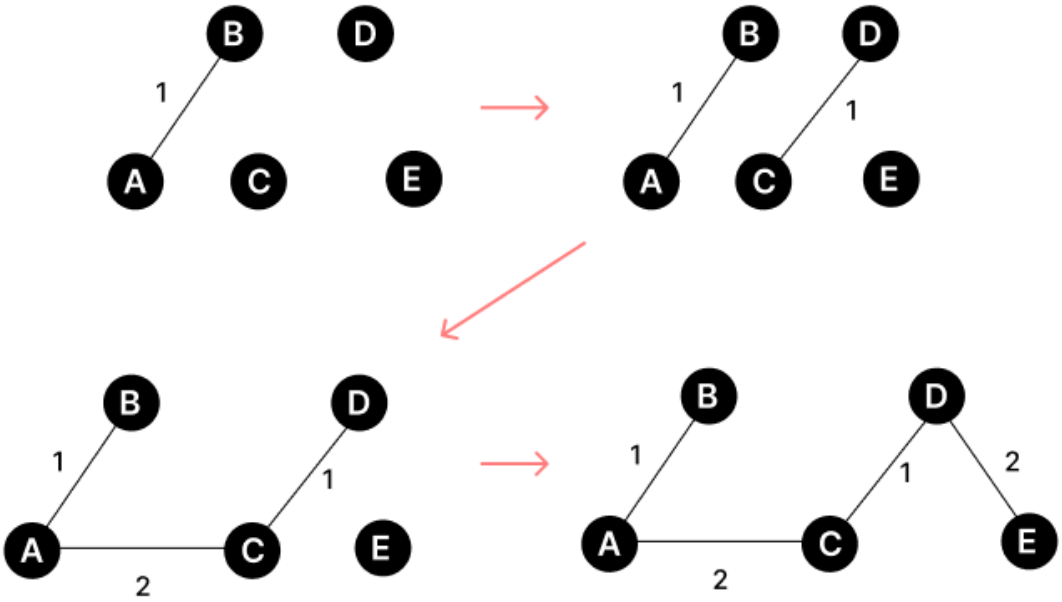
```
// Initialize the Disjoint-Set Union data structure
function makeSet(x):
    x.parent = x
    x.rank = 0

// Find the representative of the set containing x
function findSet(x):
    if x.parent != x:
        x.parent = findSet(x.parent) // Path compression heuristic
    return x.parent

// Merge the sets containing x and y
function unionSets(x, y):
    xRoot = findSet(x)
    yRoot = findSet(y)
    if xRoot == yRoot:
        return
    if xRoot.rank < yRoot.rank:
        xRoot.parent = yRoot
    else if xRoot.rank > yRoot.rank:
        yRoot.parent = xRoot
    else:
        yRoot.parent = xRoot
        xRoot.rank = xRoot.rank + 1 // Union-by-rank heuristic
```

12. “Kruskal's algorithm grows a forest of trees and eventually forms the MST.” - Explain this statement with an example

Answer:
Kruskal's algorithm is a greedy algorithm that finds the minimum spanning tree (MST) of a weighted graph by growing a forest of trees and adding edges that do not form a cycle. **The algorithm sorts the edges in ascending order of weight and repeatedly adds the smallest edge until all vertices are connected.**



Kruskal Algorithm:



In general, **Kruskal's algorithm grows a forest of trees and gradually merges them together until there is only one tree left, which represents the minimum spanning tree of the graph.**

13. What are the benefits of a good Hash Function?

Answer:
A good hash function has several benefits, which are as follows:

- **Speed**: A good hash function is very fast to compute.
- **Minimal duplication of output values**: A good hash function minimizes duplication of output values (collisions).
- **Uniform distribution**: A good hash function uniformly distributes the data across the entire set of possible hash values.
- **Deterministic**: A hash function always generates the same hash value for a given input.
- **Bijective**: A good hash function is bijective not to lose information, where possible.
- **Secure access to data**: A good hash function provides secure access to the data.
- **Constant time for searching, insertion, and deletion operations**: Hash tables are more efficient than search trees or other data structures.

A good hash function ensures that the hash function application will behave as well as if it were using a random function, for any distribution of the input data. It will, however, have more collisions than perfect hashing and may require more operations than a special-purpose hash function.

14. **Which data structure is best suited for implementing Chaining in a HashTable: Binary Search Tree or Linked List? Explain with time complexity comparisons between the both for operations such as Insertion, Search, and Deletion**.

    Answer:
    Chaining in a HashTable is a technique where instead of storing multiple values with the same hash key in the same array index, we use a data structure to store these values in a linked list or some other structure.

    Here, when implementing Chaining in a HashTable, Linked List is typically a better data structure choice over Binary Search Tree.
    Let's compare the time complexity of common operations for both data structures:

    Linked List:
    - Insertion: O(1) [average case], O(n) [worst case]
    - Search: O(n) [worst case]
    - Deletion: O(n) [worst case]

    Binary Search Tree:
    - Insertion: O(log n) average case, O(n) worst case
    - Search: O(log n) average case, O(n) worst case
    - Deletion: O(log n) average case, O(n) worst case

    As we can see, for HashTable chaining, **Linked List provides a more consistent performance with a worst-case time complexity of O(n) for all operations, while Binary Search Tree can have a worst-case time complexity of O(n) when the tree is unbalanced.**

    Therefore, it is better to use a Linked List for chaining in a HashTable as it provides a simpler implementation and better performance in the worst-case scenario.

15. Explain the working of the Rolling Hash Function in the Rabin-Karp algorithm using a suitable numerical example.

Answer:
Suppose, our pattern and string are only composed of digits. For example,

T = “723523” And P = “352”

In that case, the hash value of P = 2 + 10 * (5 + 10 * 3) = 352 [Using Horner’s rule]

Rolling hash value for the first three characters of T,

$t_0 = 3 + 10 * (2 + 10 * 7) = 723$ [Using Horner’s rule]

Now, we do not need to calculate the rolling hash value for T[2:4] using Horner’s rule. We can calculate it easily in the following way.

$t_1 = (723 - 7* 100) * 10 + 5 = 235.$

This takes constant time.

Since, each subsequent hash subtracts the first character’s ascii value and adds the next character’s ascii value the hash function is called rolling hash, a.k.a, the hash rolls forward in the string.

16. **What do you understand by edge relaxation? Let d(v) be the shortest path estimate from node s to node v and δ(s,v) be the shortest path value from node s to node v. When does it happen that d[v] = δ(s, v)?**

    Answer:
    Edge relaxation is a process used in shortest-path algorithms, such as Dijkstra's algorithm, to update the shortest path estimate of a vertex by considering its adjacent vertices.

    Condition for edge relaxation:

    $$d[v] = \min\{ d[v] \ , \ d[s] + \delta(s, v) \}$$

    Here is the question, the condition d[v] = δ(s, v) happens when the shortest path estimate of vertex v is equal to the actual shortest path value from s to v. This condition is satisfied when all the edges in the shortest path from s to v have been relaxed.
    In other words, the shortest path estimate of v is updated to the actual shortest path value when all the edges in the shortest path have been considered and relaxed.

17. What is Secondary Clustering? When might a hash table face this suffering?

Answer:
Secondary clustering is a problem that can occur in hash tables when different keys hash to the same location and follow the same probing sequence.

Secondary clustering can occur when the load factor of the hash table is high. The load factor is the ratio of the number of elements in the hash table to the size of the hash table. **When the load factor is high, the probability of collisions increases, and the length of the probing chains also increases, leading to secondary clustering.**

To avoid secondary clustering, hash tables often use a technique called "double hashing" where a secondary hash function is used to calculate the step size for probing in case of collisions, which helps to distribute keys more evenly and reduce clustering.

18. **Write a pseudocode for the Disjoint set data structure that prints all the elements that are present in the set where an element E is present. You can assume that the disjoint set has N elements from 1 to N and Make-set, Find-set and Union operations are already implemented.**

    Answer:
    Here is the pseudocode for the Disjoint set data structure that prints all the elements that are present in the set where an element E is present:

```
function printSetElements(E):
    root = Find-Set(E)
    for i = 1 to N:
        if Find-Set(i) == root:
            print i
```

    The function takes an element E as input and finds the root of the set where E is present using the Find-Set operation. It then iterates over all the elements from 1 to N and checks if the root of the current element is the same as the root of E. If so, it prints the element. This prints all the elements that are present in the set where E is present.

19. **Suppose using some unknown algorithm, you have obtained the following shortest path from a source s to a destination t: s, d, f, g, h, j, k, f, r, t. The graph contains all positive edge weights. Even though you do not know what algorithm has been used, mention why this shortest path is incorrect.**

    Answer:
    The shortest path s, d, f, g, h, j, k, f, r, t is incorrect because **f has appeared multiple times which implies the path contains a cycle**. The shortest path in a graph cannot contain a cycle because the length of the path will be more which can be reduced only by removing the cycle. This contradicts the definition of the shortest path, which is the path with the minimum total weight.

    To obtain the correct shortest path, we have to omit the cycle. For example, we can remove the path that is making the cycle ie f to f path, **f, g, h, j, k, f** then the shortest path from s to t will be,

    s, d, f, r, t

20. **What is Primary Clustering? Which Collision Resolution technique resolves this suffering?**

    Answer:
    Primary clustering is a problem of collision resolution schemes such as linear probing, **which probes the next location in the table if the current location is occupied.**

    To resolve primary clustering, collision resolution techniques such as **Quadratic probing** can be used.

    Quadratic probing is a collision resolution technique that resolves primary hashing. It solves primary clustering in hash tables by using a probing sequence that increments quadratically rather than linearly, which helps to evenly distribute keys that hash to the same index.